Experiment 7

Date : 16-03-2026

Name : M.Sreekar
Roll no : 31
Reg : 127156098

Support Vector Machine (Non linear)

Description :

Support Vector Machine is a supervised learning algorithm that finds the optimal hyperplane which best separates different classes in the feature space. It aims to maximize the margin between the closest points of different classes (support vectors).

Algorithm :

Step-by-step Approach:

1. Load and explore the dataset

- Import libraries and load the dataset (e.g., Iris).
- Inspect features and check for missing values.

2. Preprocess data: Convert labels to (-1, 1) format

- Map target classes to $y \in \{-1, 1\}$ for the SVM hinge loss function.

3. Split data into features (X) and target (y)

- Separate independent variables (X) and the target (y).
- Partition into training and testing sets.

4. Standardize features to normalize data

- Scale features to mean 0 and variance 1 to ensure faster convergence.

5. Implement Linear SVM from scratch:

- **Initialize** weights (w) and bias (b) to zero.
- **For each epoch**, iterate through all samples.
- **Check margin condition:** $y_i(w \cdot x_i + b) \geq 1$.
- **Update weights** using gradient descent:
  - If correctly classified: $w = w - lr(2\lambda w)$
  - If misclassified: $w = w - lr(2\lambda w - x_i y_i)$
- **Update bias:** $b = b + lr \cdot y_i$ (for misclassified samples).

6. Make predictions

- Use the decision function: $\text{sign}(w \cdot x + b)$.

7. Evaluate model performance

- Calculate **accuracy**, **classification report**, and **confusion matrix**.

8. Visualize results

- Plot the decision boundary and the margins.

Data Description :

Dataset : https://archive.ics.uci.edu/ml/machine-learning-databases/balan ce-scale/balance-scale.data

Data description: It can process binary classification only

Ex - 7- SVM (Non Linear Data)

Code - LR

Ex : 7

SUPPORT VECTOR MACHINE (SVM) CLASSIFIER

Objective:

To implement Support Vector Machine (SVM) algorithm for classification and understand its working principles, kernel tricks, and hyperparameter tuning.

Dataset:

Credit card Fraud Dataset (Kaggle)

Algorithm Overview:

Support Vector Machine is a supervised learning algorithm that finds the optimal hyperplane which best separates different classes in the feature space. It aims to maximize the margin between the closest points of different classes (support vectors).

Step-by-Step Approach for Credit Card Fraud Detection

Step 1: Import Required Libraries

- Import essential libraries: `NumPy`, `Pandas`, `Matplotlib`, `Seaborn`.
- Import `scikit-learn` modules for SVM, data preprocessing, and evaluation metrics.

Step 2: Load and Explore the Dataset

- Load the Credit Card Fraud dataset.
- Perform initial exploration: check dataset shape, class distribution, and missing values.

Step 3: Balance the Dataset

- Handle class imbalance by downsampling the majority class (normal transactions).
- Create a balanced dataset (e.g., 984 normal and 492 fraudulent transactions).

Step 4: Split and Scale the Data

- Separate features (X) and target (y).
- Split into training (80%) and testing (20%) sets.
- Apply `StandardScaler` to normalize features for optimal SVM performance.

Step 5: Train SVM with Different Kernels

- Train SVM classifiers using different kernels:

- **Linear**
- **Polynomial**
- **RBF (Radial Basis Function)**
- **Sigmoid**

Step 6: Evaluate and Compare Kernel Performance

- Compare accuracy scores across different kernels.
- Use a **bar chart** visualization to identify the best-performing kernel.

Step 7: Detailed Analysis of Best Model

- Select the top-performing kernel.
- Generate a **Classification Report** (Precision, Recall, F1-Score).
- Plot a **Confusion Matrix** to analyze false positives and false negatives.

Step 1: Import Required Libraries

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.decomposition import PCA

print("□ Libraries imported successfully!")
```

```
□ Libraries imported successfully!
```

Explanation:

- **numpy, pandas**: For data manipulation and analysis
- **matplotlib, seaborn**: For data visualization
- **sklearn**: Provides SVM implementation and evaluation metrics
- **PCA**: For dimensionality reduction (helps in visualization)
- **GridSearchCV**: For hyperparameter tuning

Step 2: Load and Explore the Dataset

```
# Step 2: Load the dataset
file_path = r"E:\Z_Arise\ML_Lab\creditcard.csv"

df = pd.read_csv(file_path)
```

5

```
df.head()
```


```
{"columns":[{"name":"index","rawType":"int64","type":"integer"},
{"name":"Time","rawType":"float64","type":"float"},
{"name":"V1","rawType":"float64","type":"float"},
{"name":"V2","rawType":"float64","type":"float"},
{"name":"V3","rawType":"float64","type":"float"},
{"name":"V4","rawType":"float64","type":"float"},
{"name":"V5","rawType":"float64","type":"float"},
{"name":"V6","rawType":"float64","type":"float"},
{"name":"V7","rawType":"float64","type":"float"},
{"name":"V8","rawType":"float64","type":"float"},
{"name":"V9","rawType":"float64","type":"float"},
{"name":"V10","rawType":"float64","type":"float"},
{"name":"V11","rawType":"float64","type":"float"},
{"name":"V12","rawType":"float64","type":"float"},
{"name":"V13","rawType":"float64","type":"float"},
{"name":"V14","rawType":"float64","type":"float"},
{"name":"V15","rawType":"float64","type":"float"},
{"name":"V16","rawType":"float64","type":"float"},
{"name":"V17","rawType":"float64","type":"float"},
{"name":"V18","rawType":"float64","type":"float"},
{"name":"V19","rawType":"float64","type":"float"},
{"name":"V20","rawType":"float64","type":"float"},
{"name":"V21","rawType":"float64","type":"float"},
{"name":"V22","rawType":"float64","type":"float"},
{"name":"V23","rawType":"float64","type":"float"},
{"name":"V24","rawType":"float64","type":"float"},
{"name":"V25","rawType":"float64","type":"float"},
{"name":"V26","rawType":"float64","type":"float"},
{"name":"V27","rawType":"float64","type":"float"},
{"name":"V28","rawType":"float64","type":"float"},
{"name":"Amount","rawType":"float64","type":"float"},
{"name":"Class","rawType":"int64","type":"integer"}],"ref":"b4f4563d-
45c7-4fe4-8aa8-5c215639da8a","rows":[["0","0.0","-1.3598071336738","-
0.0727811733098497","2.53634673796914","1.37815522427443","-
0.338320769942518","0.462387777762292","0.239598554061257","0.09869790
12610507","0.363786969611213","0.0907941719789316","-
0.551599533260813","-0.617800855762348","-0.991389847235408","-
0.311169353699879","1.46817697209427","-
0.470400525259478","0.207971241929242","0.0257905801985591","0.4039929
60255733","0.251412098239705","-
0.018306777944153","0.277837575558899","-
0.110473910188767","0.0669280749146731","0.128539358273528","-
0.189114843888824","0.133558376740387","-
0.0210530534538215","149.62","0"],
["1","0.0","1.19185711131486","0.26615071205963","0.16648011335321","0
.448154078460911","0.0600176492822243","-0.0823608088155687","-
0.0788029833323113","0.0851016549148104","-0.255425128109186","-
```

0.166974414004614","1.61272666105479","1.06523531137287","0.4890950158
9608","-0.143772296441519","0.635558093258208","0.463917041022171","-
0.114804663102346","-0.183361270123994","-0.145783041325259","-
0.0690831352230203","-0.225775248033138","-
0.638671952771851","0.101288021253234","-
0.339846475529127","0.167170404418143","0.125894532368176","-
0.0089830991432281","0.0147241691924927","2.69","0"],["2","1.0","-
1.35835406159823","-
1.34016307473609","1.77320934263119","0.379779593034328","-
0.503198133318193","1.80049938079263","0.791460956450422","0.247675786
588991","-
1.51465432260583","0.207642865216696","0.624501459424895","0.066083685
268831","0.717292731410831","-0.165945922763554","2.34586494901581","-
2.89008319444231","1.10996937869599","-0.121359313195888","-
2.26185709530414","0.524979725224404","0.247998153469754","0.771679401
917229","0.909412262347719","-0.689280956490685","-
0.327641833735251","-0.139096571514147","-0.0553527940384261","-
0.0597518405929204","378.66","0"],["3","1.0","-0.966271711572087","-
0.185226008082898","1.79299333957872","-0.863291275036453","-
0.0103088796030823","1.24720316752486","0.23760893977178","0.377435874
652262","-1.38702406270197","-0.0549519224713749","-
0.226487263835401","0.178228225877303","0.507756869957169","-
0.28792374549456","-0.631418117709045","-1.0596472454325","-
0.684092786345479","1.96577500349538","-1.2326219700892","-
0.208037781160366","-0.108300452035545","0.0052735967825345","-
0.190320518742841","-1.17557533186321","0.647376034602038","-
0.221928844458407","0.0627228487293033","0.0614576285006353","123.5","
0"],["4","2.0","-
1.15823309349523","0.877736754848451","1.548717846511","0.403033933955
121","-0.407193377311653","0.0959214624684256","0.592940745385545","-
0.270532677192282","0.817739308235294","0.753074431976354","-
0.822842877946363","0.53819555014995","1.3458515932154","-
1.11966983471731","0.175121130008994","-0.451449182813529","-
0.237033239362776","-
0.0381947870352842","0.803486924960175","0.408542360392758","-
0.0094306971323291","0.79827849458971","-
0.137458079619063","0.141266983824769","-
0.206009587619756","0.502292224181569","0.219422229513348","0.21515314
7499206","69.99","0"]],"shape":{"columns":31,"rows":5}}

```
print("\n" + "="*60)
print("CREDIT CARD FRAUD DATASET")
print("="*60)
print(f"Dataset shape: {df.shape}")
print(f"\nFirst 5 rows:")
print(df.head())

print(f"\nClass distribution:")
print(df['Class'].value_counts())
```

7

```
print(f"\nFraudulent transactions: {df['Class'].sum()}
({df['Class'].mean()*100:.4f}%)")
```

```
============================================================
CREDIT CARD FRAUD DATASET
============================================================
Dataset shape: (284807, 31)

First 5 rows:
   Time        V1        V2        V3        V4        V5        V6
V7  \
0   0.0 -1.359807 -0.072781  2.536347  1.378155 -0.338321  0.462388
0.239599
1   0.0  1.191857  0.266151  0.166480  0.448154  0.060018 -0.082361 -
0.078803
2   1.0 -1.358354 -1.340163  1.773209  0.379780 -0.503198  1.800499
0.791461
3   1.0 -0.966272 -0.185226  1.792993 -0.863291 -0.010309  1.247203
0.237609
4   2.0 -1.158233  0.877737  1.548718  0.403034 -0.407193  0.095921
0.592941

         V8        V9  ...       V21       V22       V23       V24
V25  \
0  0.098698  0.363787  ... -0.018307  0.277838 -0.110474  0.066928
0.128539
1  0.085102 -0.255425  ... -0.225775 -0.638672  0.101288 -0.339846
0.167170
2  0.247676 -1.514654  ...  0.247998  0.771679  0.909412 -0.689281 -
0.327642
3  0.377436 -1.387024  ... -0.108300  0.005274 -0.190321 -1.175575
0.647376
4 -0.270533  0.817739  ... -0.009431  0.798278 -0.137458  0.141267 -
0.206010

        V26       V27       V28  Amount  Class
0 -0.189115  0.133558 -0.021053  149.62      0
1  0.125895 -0.008983  0.014724    2.69      0
2 -0.139097 -0.055353 -0.059752  378.66      0
3 -0.221929  0.062723  0.061458  123.50      0
4  0.502292  0.219422  0.215153   69.99      0

[5 rows x 31 columns]

Class distribution:
Class
0    284315
1       492
Name: count, dtype: int64
```

```
Fraudulent transactions: 492 (0.1727%)
```

```python
# Check for missing values
print("\n" + "="*50)
print("MISSING VALUES CHECK")
print("="*50)
print(df.isnull().sum())
```

```
==================================================
MISSING VALUES CHECK
==================================================
Time      0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
V10       0
V11       0
V12       0
V13       0
V14       0
V15       0
V16       0
V17       0
V18       0
V19       0
V20       0
V21       0
V22       0
V23       0
V24       0
V25       0
V26       0
V27       0
V28       0
Amount    0
Class     0
dtype: int64
```

Explanation:

- The Wine Quality dataset contains 1599 samples with 12 features
- Features are chemical properties of wine (all numerical)
- Target variable 'quality' ranges from 3 to 8

- No missing values in the dataset (good for SVM)
- Features have different scales (will need normalization)

Step 3: Data Preprocessing for SVM

```
# Step 3: Data Preprocessing
X = df.drop('Class', axis =1)
y = df['Class']

# Simple downsampling of majority class (for faster training)
# This is a simple approach - for production, use SMOTE or other
techniques
fraud = df[df['Class'] == 1]
normal = df[df['Class'] == 0].sample(n=len(fraud) * 2,
random_state=42)

df_balanced = pd.concat([fraud, normal])
X_balanced = df_balanced.drop('Class', axis=1)
y_balanced = df_balanced['Class']

print(f"\nBalanced dataset shape: {X_balanced.shape}")
print(f"Balanced class distribution:\n{y_balanced.value_counts()}")
```

```
Balanced dataset shape: (1476, 30)
Balanced class distribution:
Class
0    984
1    492
Name: count, dtype: int64
```

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X_balanced, y_balanced, test_size=0.2, random_state=42,
stratify=y_balanced)

print(f"Training set size: {X_train.shape[0]} samples")
print(f"Testing set size: {X_test.shape[0]} samples")
```

```
Training set size: 1180 samples
Testing set size: 296 samples
```

```
# Scale the features (CRITICAL for SVM)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Why Scaling is Critical for SVM:

- SVM tries to maximize the margin between classes

- Features with larger scales would dominate the distance calculation
- Standard scaling ensures all features contribute equally
- Without scaling, the SVM hyperplane would be biased

Step 5 : Non linear SVM

```
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score, GridSearchCV

kernels = ['linear', 'poly', 'rbf', 'sigmoid']
results = {}

print("\n" + "="*60)
print("SVM PERFORMANCE WITH DIFFERENT KERNELS")
print("="*60)

for kernel in kernels:
    print(f"\n Training SVM with {kernel.upper()} kernel...")

    # Create and train SVM model
    svm = SVC(kernel=kernel, random_state=42, C=1.0)
    svm.fit(X_train_scaled, y_train)

    # Make predictions
    y_pred = svm.predict(X_test_scaled)
    accuracy = accuracy_score(y_test, y_pred)

    # Store results
    results[kernel] = {
        'model': svm,
        'accuracy': accuracy,
        'predictions': y_pred
    }

    print(f"   Test Accuracy: {accuracy:.4f}")
    print(f"   Support Vectors: {svm.n_support_}")
    print(f"   Total Support Vectors: {len(svm.support_vectors_)}")

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Tuned SVM')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```

```
============================================================
SVM PERFORMANCE WITH DIFFERENT KERNELS
```

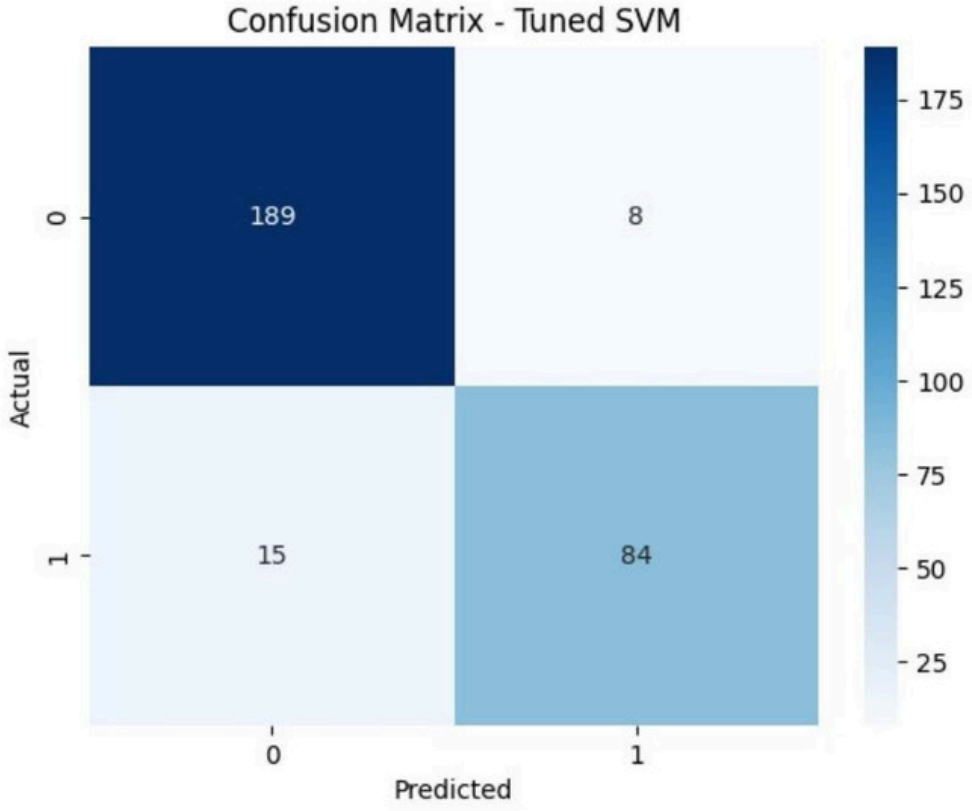
```
============================================================

 Training SVM with LINEAR kernel...
   Test Accuracy: 0.9662
   Support Vectors: [68 62]
   Total Support Vectors: 130

 Training SVM with POLY kernel...
   Test Accuracy: 0.8919
   Support Vectors: [204 186]
   Total Support Vectors: 390

 Training SVM with RBF kernel...
   Test Accuracy: 0.9493
   Support Vectors: [134 128]
   Total Support Vectors: 262

 Training SVM with SIGMOID kernel...
   Test Accuracy: 0.9223
   Support Vectors: [80 77]
   Total Support Vectors: 157
```



SVM Kernels Explained:

Kernel	Description	When to Use
Linear	Creates a straight line/hyperplane separator	When data is linearly separable
Polynomial	Uses polynomial function to create curved boundaries	For moderately complex relationships
RBF (Radial Basis Function)	Most popular, creates complex non-linear boundaries	Default choice, works well for most datasets
Sigmoid	Similar to neural network activation function	For specific patterns, less common

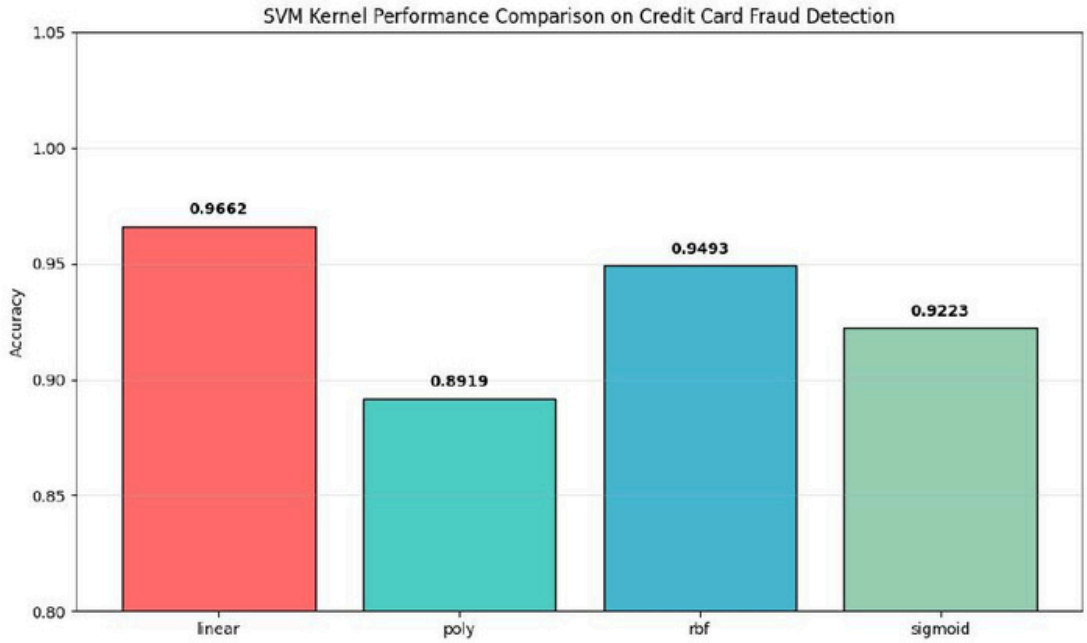
Key Observations from Visualization:

- Different kernels create different decision boundaries
- Support vectors are the critical points that define the margin
- RBF kernel often provides the most flexible boundary

```
# Accuracy comparison bar chart
plt.figure(figsize=(10, 6))
accuracies = [results[k]['accuracy'] for k in kernels]
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#96CEB4']
bars = plt.bar(kernels, accuracies, color=colors, edgecolor='black')
plt.ylabel('Accuracy')
plt.title('SVM Kernel Performance Comparison on Credit Card Fraud
Detection')
plt.ylim(0.8, 1.05)

for bar, acc in zip(bars, accuracies):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() +
0.005,
             f'{acc:.4f}', ha='center', fontweight='bold')

plt.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()
```

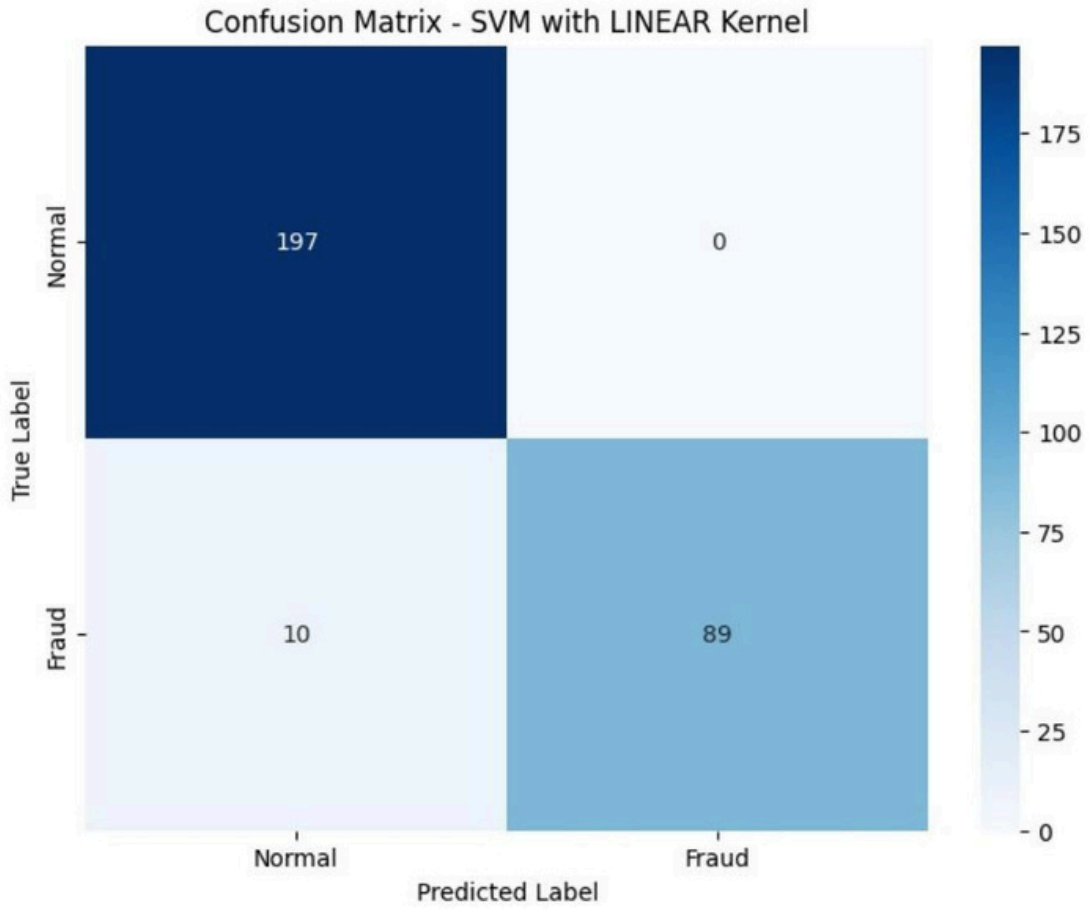



```
# Find the best kernel
best_kernel = max(results, key=lambda k: results[k]['accuracy'])
best_svm = results[best_kernel]['model']
y_pred_best = results[best_kernel]['predictions']

print("\n" + "="*60)
print(f"DETAILED EVALUATION - {best_kernel.upper()} KERNEL")
print("="*60)

# Classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred_best,
target_names=['Normal', 'Fraud']))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred_best)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Fraud'],
            yticklabels=['Normal', 'Fraud'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title(f'Confusion Matrix - SVM with {best_kernel.upper()} Kernel')
plt.show()
```

```
============================================================
DETAILED EVALUATION - LINEAR KERNEL
============================================================
```

```
Classification Report:
              precision    recall  f1-score   support

      Normal       0.95      1.00      0.98       197
       Fraud       1.00      0.90      0.95        99

    accuracy                           0.97       296
   macro avg       0.98      0.95      0.96       296
weighted avg       0.97      0.97      0.97       296
```